RISC-V Memory Model

Memory consistency model

- A **memory consistency model** is a set of rules that defines the order in which memory operations (such as reads and writes) appear to execute from the perspective of different threads or processors in a multiprocessor system. It dictates how and when the changes made by one processor to the memory are visible to other processors.

- The RISC-V ISA (Reduced Instruction Set Computer - Five Instruction Set Architecture) by default uses a memory model called “RVWMO” (RISC-V Weak Memory Ordering).

RVWMO

- The RVWMO memory model is defined in terms of the global memory order, a total ordering of the memory accesses produced by all harts. In general, a multithreaded program will have many different possible executions, and each execution will have its its own corresponding global memory order.
- The global memory order is defined in terms of the primitive load(s) and/or store(s) generated by each memory instruction.
- RISC-V has chosen the RVWMO memory model, a variant of release consistency. This places it in between the two extremes of the memory model spectrum. It is neither as strict as Sequential Consistency (SC) nor as relaxed as some of the weakest models.
- **Release Consistency (RC)**: Divides operations into acquire and release, allowing greater flexibility by enforcing ordering only at synchronization points.

Litmus tests

- Litmus tests are small, focused programs designed to test specific behaviors of a memory model. In the context of memory consistency models, litmus tests help illustrate how different memory operations might interact under various conditions.

- A litmus test source has three main sections:

  1. The initial state defines the initial values of registers and memory locations. Initialisation to zero may be omitted.

  2. The code section defines the code to be run concurrently — above there are two threads.

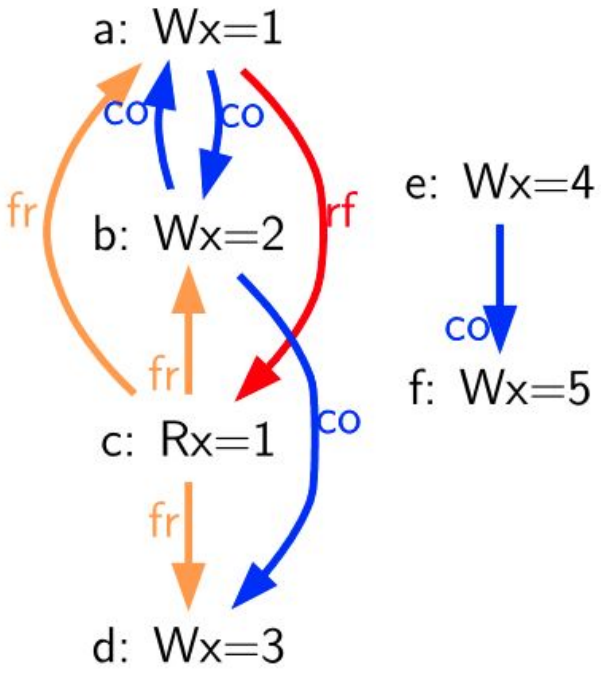
  3. The final condition applies to the final values of registers and memory locations.

```
X86 SB
"Fre PodWR Fre PodWR"
{ x=0; y=0; }
P0          | P1;
MOV [x],$1  | MOV [y],$1 ;
MOV EAX,[y] | MOV EAX,[x] ;
locations [x;y;]
exists (0:EAX=0 /\ 1:EAX=0)
```

Edge	Full Name (and explanation)
rf	Reads From (from each store to the loads that return a value written by that store)
co	Coherence (a total order on the stores to each address)
fr	From-Reads (from each load to co-successors of the store from which the load returned a value)
ppo	Preserved Program Order
fence	Orderings enforced by a FENCE instruction
addr	Address Dependency
ctrl	Control Dependency
data	Data Dependency

Sample Litmus test

	Hart 0		Hart 1
	⋮		⋮
	li t1, 1		li t4, 4
(a)	sw t1,0(s0)	(e)	sw t4,0(s0)
	⋮		⋮
	li t2, 2		
(b)	sw t2,0(s0)		
	⋮		⋮
(c)	lw a0,0(s0)		
	⋮		⋮
	li t3, 3		li t5, 5
(d)	sw t3,0(s0)	(f)	sw t5,0(s0)
	⋮		⋮



Preserved Program Order (PPO) and Global Memory Order (GMO)

Preserved Program Order (PPO) represents a subset of the program order that must be respected within the Global Memory Order (GMO). In other words, certain operations from the same hart (hardware thread) must appear in the same order to other harts as they do in the program order.

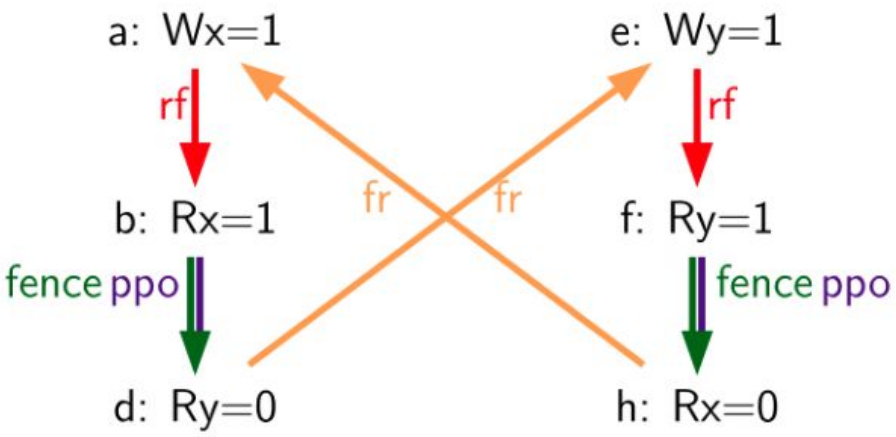
The GMO is the conceptual order in which memory operations (loads and stores) appear to execute globally. It represents how memory operations from different harts are interleaved into a single, globally agreed-upon sequence.

Load value axiom

- Each byte of each load *i* returns the value written to that byte by the store that is the latest in global memory order among the following stores:
  1. Stores that write that byte and that precede i in the global memory order
  2. Stores that write that byte and that precede i in program order
- Among all stores (from both global and program order considerations), the store with the latest write to the byte in the global memory order is selected. The load operation returns the value written by this store.
- In many memory models, preserved program order does not require a strict ordering between a store followed by a load to the same address within the same hart.This flexibility arises because of the common use of **store buffers** in modern processors.
- **Store Buffers** are hardware mechanisms that temporarily hold store operations before they are written to globally visible memory. This allows the processor to continue executing subsequent instructions without waiting for the store to complete.
- When a load follows a store to the same address, the load can bypass waiting for the store to write to memory by **forwarding** the value directly from the store buffer. This is an optimization technique improves performance by reducing the latency that would otherwise occur if the load had to wait.

Store buffer forwarding litmus test

Hart 0	Hart 1
li t1, 1	li t1, 1
(a) sw t1,0(s0)	(e) sw t1,0(s1)
(b) lw a0,0(s0)	(f) lw a2,0(s1)
(c) fence r,r	(g) fence r,r
(d) lw a1,0(s1)	(h) lw a3,0(s0)
Outcome: `a0=1`, `a1=0`, `a2=1`, `a3=0`	



Atomicity Axiom

Statement : If r and w are paired load and store operations generated by aligned LR and SC instructions in a hart h, s is a store to byte x, and r returns a value written by s, then s must precede w in the global memory order, and there can be no store from a hart other than h to byte x following s and preceding w in the global memory order.

The instructions LR (Load-Reserved) and SC (Store-Conditional) are used to implement atomic read-modify-write sequences. The LR loads a value from memory and reserves the memory address, while the SC attempts to store a new value back to that address.

LR/SC pairs and AMOs

RISC-V contains two types of atomics:

1. **LR/SC pairs**

   LR/SC pairs involve two separate operations:

   a. First, a load (LR) that reads a value from memory and establishes a reservation on the memory location
   b. The SC then checks if the reservation is still valid (i.e., no other hart has written to the memory location since the LR). If the reservation is valid, the SC writes the new value to memory and succeeds; if not, it fails and the operation must be retried

2. **AMOs**
   a. These are single atomic operations that perform a read-modify-write sequence directly in memory
   b. An AMO reads a value from memory, applies an operation (e.g., addition), and writes the result back to memory—all as a single atomic transaction.
   c. Examples include atomic addition (amoadd), bitwise operations, and other

LR/SC instructions examples

(a) lr.d a0, 0(s0)	(a) lr.d a0, 0(s0)	(a) lr.w a0, 0(s0)	(a) lr.w a0, 0(s0)
(b) sd t1, 0(s0)	(b) sw t1, 4(s0)	(b) sw t1, 4(s0)	(b) sw t1, 4(s0)
(c) sc.d t3, t2, 0(s0)	(c) sc.d t3, t2, 0(s0)	(c) sc.w t3, t2, 0(s0)	(c) addi s0, s0, 8
(d) sc.w t3, t2, 8(s0)			

Progress axiom

Statement : No memory operation may be preceded in the global memory order by an infinite sequence of other memory operations.

Explanation : Stores from one hart (processor core) will eventually become visible to other harts within a finite time. This guarantees that memory operations are not indefinitely delayed and that data will eventually be visible to other parts of the system.

Usage:

1. Visibility of stores by forwarding
2. Avoiding Infinite Delay

RVWMO rules (Rule 1-13)

Overlapping-Address orderings

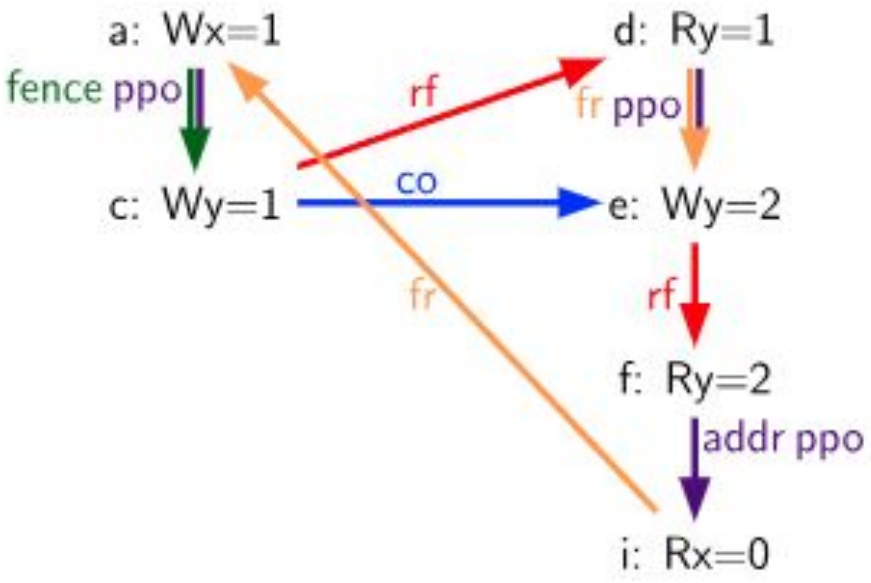
Rule 1: Store ordering [ppo:->st]

- **Statement**: If b is a store and a and b access overlapping memory addresses, then a (if it is a load) cannot be reordered with b.
- **Explanation**: A store operation b to a memory address must be visible to any subsequent load operation a accessing the same address. This means that if a store to an address is performed, any subsequent load to that address must see the value written by that store, preserving the ordering of stores relative to loads.

Example of Rule 1:

	Hart 0		Hart 1
	li t1, 1		li t2, 2
(a)	sw t1,0(s0)	(d)	lw a0,0(s1)
(b)	fence w, w	(e)	sw t2,0(s1)
(c)	sw t1,0(s1)	(f)	lw a1,0(s1)
		(g)	xor t3,a1,a1
		(h)	add s0,s0,t3
		(i)	lw a2,0(s0)

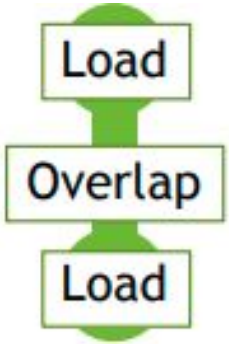
Outcome: a0=1, a1=2, a2=0



Rule 2: Load-Load Ordering [ppo:rdw]

- **Statement :** a and b are loads, x is a byte read by both a and b, there is no store to x between a and b in program order, and a and b return values for x written by different memory operations

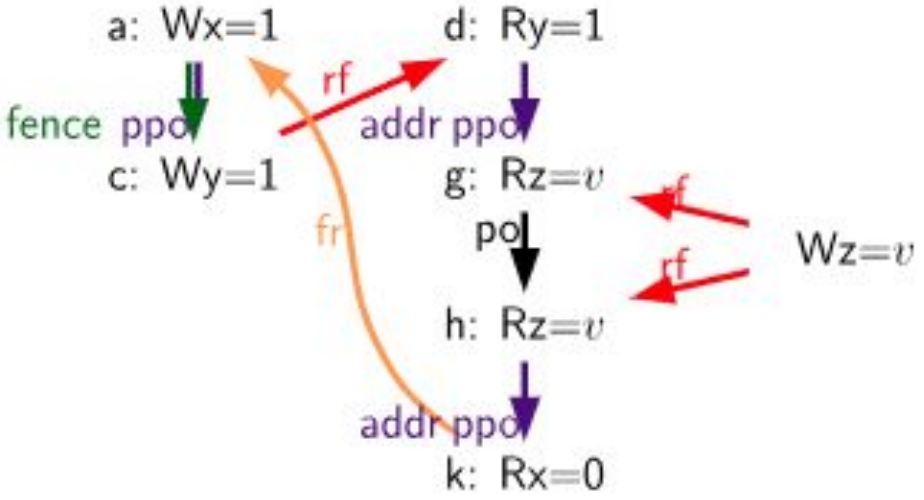
- **Explanation :**
  - Both a and b are loading from the same memory location s0.
  - In a straightforward execution, a and b return the same value since they read from the same address.
  - However, if a returns a different value than b, This could happen in a system with a relaxed memory consistency model, where loads might see different values depending on how memory operations are interleaved and executed.
  - This rule ensures the principle of Coherence for Read-Read pairs (CoRR), means a younger load cannot return a value older than what was returned by an older load.



Example of Rule 2:

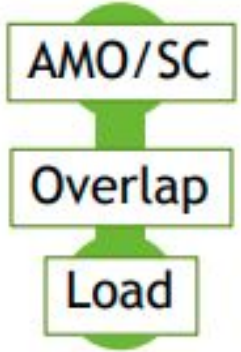
	Hart 0		Hart 1
	li t1, 1	(d)	lw a0,0(s1)
(a)	sw t1,0(s0)	(e)	xor t2,a0,a0
(b)	fence w, w	(f)	add s4,s2,t2
(c)	sw t1,0(s1)	(g)	lw a1,0(s4)
		(h)	lw a2,0(s2)
		(i)	xor t3,a2,a2
		(j)	add s0,s0,t3
		(k)	lw a3,0(s0)

Outcome: a0=1, a1=v, a2=v, a3=0



Rule 3: AMO/SC Ordering [ppo:amoforward]

Statement : If a is an AMO (Atomic Memory Operation) or SC (Store Conditional) instruction, and b is a load that returns a value written by a.



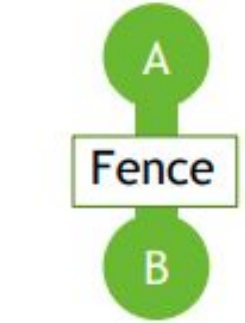
Explanation: This rule ensures that if a load operation returns a value written by an AMO or SC instruction, the AMO or SC operation must have occurred before the load in the global memory order.

Rule 4: Fences [ppo:fence]

Statement : There is a FENCE instruction that orders a before b (fence [r][w][i][o],[r][w][i][o])

Explanation:

- Specifically, it ensures that all memory operations (loads and stores) that appear before the FENCE instruction in program order are observed to occur before those that appear after the FENCE in global memory order.
- This helps synchronize memory operations, especially in a multi-core or multi-threaded environment.



with pr/pw/sr/sw
set appropriately

FENCE Bits and Their Function

31 28	27	26	25	24	23	22	21	20	19 15	14 12	11 7	6 0
fm	PI	PO	PR	PW	SI	SO	SR	SW	rs1	funct3	rd	opcode
4	1	1	1	1	1	1	1	1	5	3	5	7
FM	predecessor				successor				0	FENCE	0	MISC-MEM

The FENCE instruction includes several control bits that allow for fine-tuning of its ordering behavior. These bits restrict which memory operations are included in the ordering constraints

PR: previous reads.

PW: previous writes.

PI: previous I/O reads.

PO: previous I/O writes.

SR: subsequent reads

SW: subsequent writes

SI: subsequent I/O reads

SO: subsequent I/O writes

Non-Trivial Combinations of PR, PW, SR, and SW Bits

The FENCE encoding currently has nine non-trivial combinations of the four bits PR, PW, SR, and SW, plus one extra encoding FENCE.TSO

1. **FENCE RW, RW** (PR=1, PW=1, SR=1, SW=1)
2. **FENCE RW, W** (PR=1, PW=1, SR=, SW=1)
3. **FENCE R, RW** (PR=1, PW=0, SR=1, SW=1)
4. **FENCE R, R** (PR=1, PW=0, SR=1, SW=0)
5. **FENCE W, W** (PR=0, PW=1, SR=0, SW=1)
6. **FENCE W, RW** (PR=0, PW=1, SR=1, SW=1)
7. **FENCE RW, R** (PR=1, PW=1, SR=1, SW=0)
8. **FENCE W, R** (PR=0, PW=1, SR=1, SW=0)
9. **FENCE R, W** (PR=1, PW=0, SR=0, SW=1)

Of the ten non-trivial options, only six are commonly used in practice:

- FENCE RW,RW
- FENCE.TSO
- FENCE RW,W
- FENCE R,RW
- FENCE R,R
- FENCE W,W

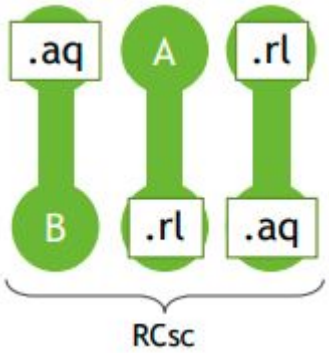
FENCE.TSO encoding can be used to map acquire and release semantics to ensure that all memory operations before the fence are visible to all operations that follow it. This helps in implementing a memory model similar to TSO (Total Store Order), where stores are globally ordered and loads can be reordered with respect to stores.

Explicit Synchronization

Rule 5: acquire annotation [ppo:acquire]

- **Statement :** An acquire operation ensures that all memory operations following it in program order (i.e., in the program's instruction sequence) will also follow it in the global memory order (the order observed by all processors in a system)

- Acquire ordering can be enforced in one of two ways:
  - with an **acquire annotation,** which enforces ordering with respect to just the synchronization variable itself. This method is often used with atomic operations or specific synchronization primitives
  - with a **FENCE R,RW,** which enforces ordering with respect to all previous loads



Rule 6: Release Annotation [ppo:release]

- **Statement :** A release operation ensures that all memory operations preceding it in program order will also precede it in the global memory order.
- Release semantics require all loads and stores preceding the release operation in program order to also precede the release operation in the global memory order.
- Release semantics can be enforced by two ways
  - with a **release annotation** on a specific instruction (like a store)
  - using a **FENCE RW,W**, which ensures that all preceding loads and stores are completed before any subsequent stores

acquire and release annotations using atomics

```
    sd        x1, (a1)    # Arbitrary unrelated store
    ld        x2, (a2)    # Arbitrary unrelated load
    li        t0, 1       # Initialize swap value.
again:
    amoswap.w.aq t0, t0, (a0) # Attempt to acquire lock.
    bnez      t0, again   # Retry if held.
    # ...
    # Critical section.
    # ...
    amoswap.w.rl x0, x0, (a0) # Release lock by storing 0.
    sd        x3, (a3)    # Arbitrary unrelated store
    ld        x4, (a4)    # Arbitrary unrelated load
```

amoswap.w.aq : performs an atomic swap. It exchanges the value in t0 with the value in the memory location pointed to by a0(lock variable : it is zero if it free or 1 if it held by some other thread) **.aq (acquire)** suffix ensures that any loads and stores following this instruction in program order will also follow it in the global memory order.

amoswap.w.rl : releases lock , where the **.rl (release)** suffix ensures that all operations within the critical section appear before the lock release in the global memory order.

acquire and release annotations using fences

```
    sd          x1, (a1)     # Arbitrary unrelated store
    ld          x2, (a2)     # Arbitrary unrelated load
    li          t0, 1        # Initialize swap value.
again:
    amoswap.w   t0, t0, (a0) # Attempt to acquire lock.
    fence       r, rw        # Enforce "acquire" memory ordering
    bnez        t0, again    # Retry if held.
    # ...
    # Critical section.
    # ...
    fence       rw, w        # Enforce "release" memory ordering
    amoswap.w   x0, x0, (a0) # Release lock by storing 0.
    sd          x3, (a3)     # Arbitrary unrelated store
    ld          x4, (a4)     # Arbitrary unrelated load
```

In this case, even though the AMOSWAP does not enforce ordering with an *aq* bit, the fence nevertheless enforces that the acquire AMOSWAP appears earlier in the global memory order than all loads and stores in the critical section.

- The lock is acquired using **amoswap.w** without the **.aq** suffix. This means the atomic swap itself does not enforce any specific memory ordering
- **FENCE r,rw** ,This fence ensures that all loads (r) that appear before it in the program order will appear before any subsequent loads or stores (rw) in the global memory order.
- Before releasing the lock, a **FENCE rw,w** instruction is used. This fence ensures that all loads and stores within the critical section (rw) are completed before any stores (w) that follow the fence in program order.
- The lock is released using **amoswap.w**

Rule 7: RCsc Annotations [ppo:rcsc]

- **Statement :** A release that precedes an acquire in program order also precedes it in global memory order i.e., the RCsc (Release Consistency with Sequential Consistency) variant of release consistency

- Why RCsc annotation is preferred over Rcpc (Release Consistency with Partial Consistency) ?

RCpc annotation	RCsc annotation
The ordering between store and load operations is more relaxed.	provides stronger memory ordering guarantees.
A **store-release** operation may not immediately be visible to a **load-acquire** operation on another thread	A **store-release** operation in one thread is guaranteed to be visible to any **load-acquire** operation in another thread
In RCpc, even if a thread performs a store with release semantics, another thread performing a load with acquire semantics may not immediately observe the result of that store	In RCsc, if a thread performs a store with release semantics, another thread performing a load with acquire semantics will observe the result of that store

Rule 8: Paired Operations [ppo:pair]

Statement : This rule refers to a scenario where two operations are explicitly paired, such as in the case of Load-Reserved (LR) and Store-Conditional (SC) operations. Paired operations ensure that specific sequences of operations are treated atomically.

Explanation :

- Often used in atomic operations where the pairing of a load and store ensures that the memory location being accessed is not modified by other operations in between the pair.
- Indicates that an SC must appear after its paired LR in the global memory order. This will follow read-modify-write operation due to the inherent data dependency.
- It guarantee that the memory location remains unchanged by other harts between the paired operations.

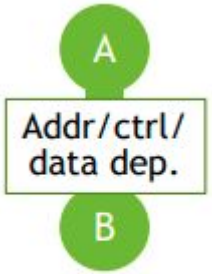
Syntactic dependencies

Rule 9: Address dependency [ppo:addr]

Statement : A memory operation b has a syntactic address dependency on a previous instruction a if the address used by b depends on the result of a

Example :

```
ld  a1, 0(s0)       # Load the value at address (s0) into register a1
xor a2, a1, a1      # XOR a1 with itself and store the result in a2 (which will be 0)
add s1, s1, a2      # Add the value in a2 (which is 0) to s1, effectively leaving s1 unchanged
ld  a5, 0(s1)       # Load the value at address (s1) into register a5
```



Here, there is a syntactic address dependency from the memory operation generated by the first instruction to the memory operation generated by the last instruction , even though a1 XOR a1 is zero and hence has no effect on the address accessed by the second load.

Rule 10 : Data Dependency [ppo:data]

Statement : The result of one instruction (A) influences the execution or outcome of another instruction (B).

Example :

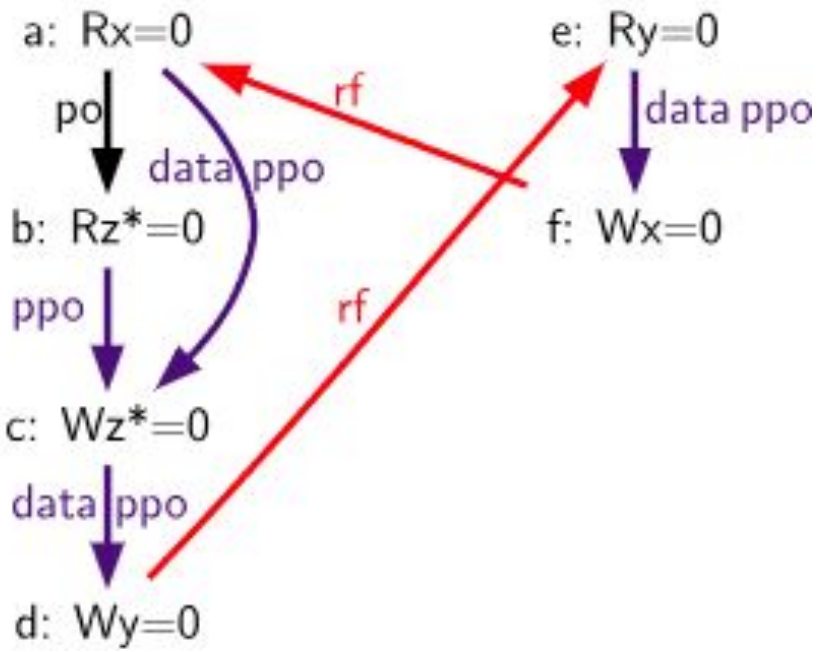
Specifically, in the context of store operations, if A produces a value that is used by B (a store operation), there is a data dependency between these two instructions.

Example of Rule 10:

Initial values: 0(s0)=1; 0(s2)=1

Hart 0		Hart 1	
(a)	ld a0,0(s0)	(e)	ld a3,0(s2)
(b)	lr a1,0(s1)	(f)	sd a3,0(s0)
(c)	sc a2,a0,0(s1)		
(d)	sd a2,0(s2)		

Outcome: a0=0, a3=0



Rule 11 : Control dependency [ppo:ctrl]

Statement : The execution of one instruction depends on the outcome of a preceding branch instruction. Also ensures that no store operation can override or affect the result of the SC operation before the SC operation has been globally observed.

Example:

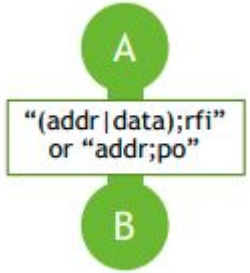
```
lw x1, 0(x2)          # (1) Load from address x2 into x1

bne x1, x0, next      # (2) Branch to 'next' if x1 is not equal to x0

sw x3, 0(x4)          # (3) Store x3 to address x4

next: sw x5, 0(x6)    # (4) Label 'next': Store x5 to address x6
```

The sw x5, 0(x6) at the next label has a control dependency on the lw x1, 0(x2) instruction. This is because the branch outcome affects whether next is reached. Even though the branch may or may not be taken, the control dependency ensures that sw x5, 0(x6) is not executed until after the branch decision is made.

Pipeline Dependencies

Rule 12 : Load with Address or Data Dependency [ppo:addrdararfi]

Statement : If b is a load instruction, and there exists some store m between a and b in program order such that m has an address or data dependency on a, and b returns a value written by m, then the ordering of a and b must be preserved in the pipeline.



Explanation:

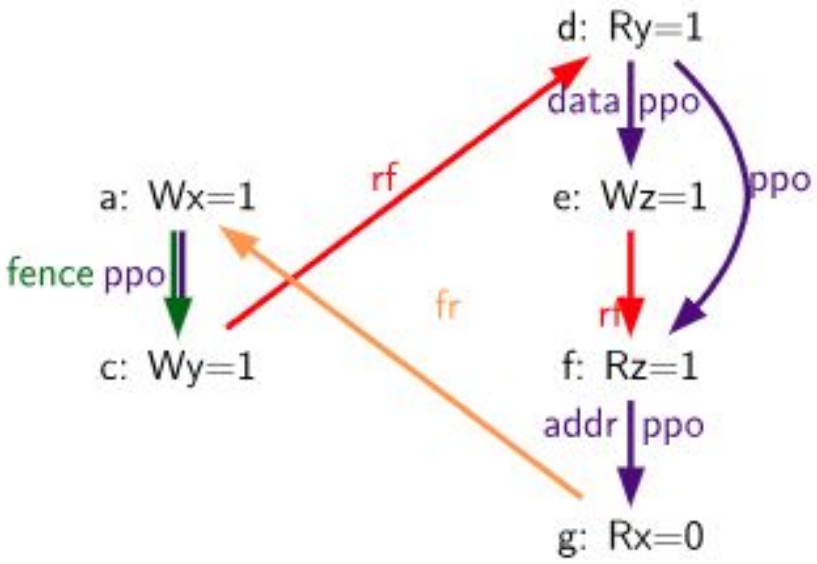
Address Dependency: The address used by m depends on the result of a.

Data Dependency: The value written by m depends on the result of a.

Pipeline Ordering: This rule ensures that the load (b) sees the value written by the store (m) and maintains the correct ordering between a and b.

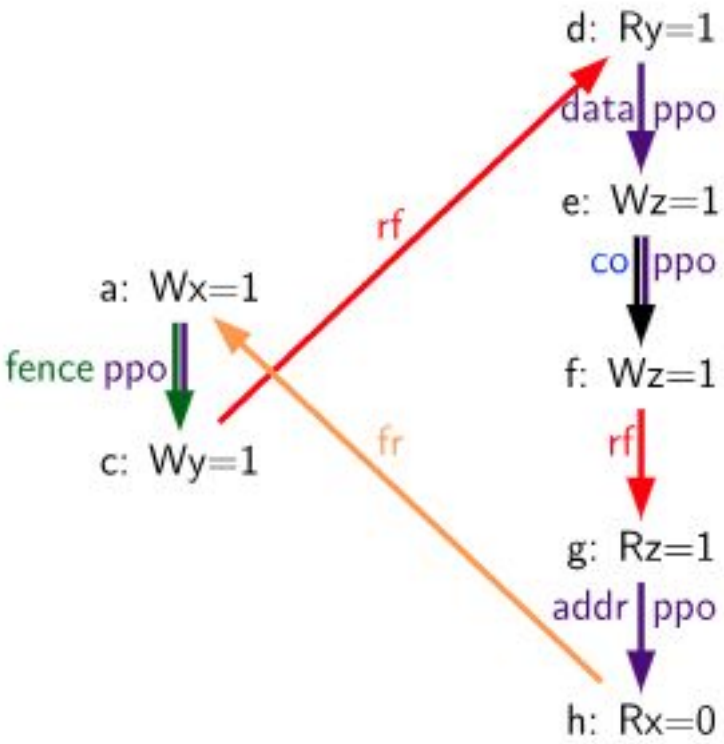
Example of Rule 12:

	Hart 0		Hart 1
	li t1, 1	(d)	lw a0, 0(s1)
(a)	sw t1,0(s0)	(e)	sw a0, 0(s2)
(b)	fence w, w	(f)	lw a1, 0(s2)
(c)	sw t1,0(s1)		xor a2,a1,a1
			add s0,s0,a2
		(g)	lw a3,0(s0)
	Outcome:		a0=1, a3=0



Because of PPO rule [ppo:addrdatarfi] and the data dependency from (d) to (e), (d) must also precede (f) in the global memory order (outcome forbidden)

	Hart 0		Hart 1
	li t1, 1		li t1, 1
(a)	sw t1,0(s0)	(d)	lw a0, 0(s1)
(b)	fence w, w	(e)	sw a0, 0(s2)
(c)	sw t1,0(s1)	(f)	sw t1, 0(s2)
		(g)	lw a1, 0(s2)
			xor a2,a1,a1
			add s0,s0,a2
		(h)	lw a3,0(s0)
	Outcome:		a0=1, a3=0



Because of the extra store between (e) and (g), (d) no longer necessarily precedes (g) (outcome permitted)

Rule 13: Store with Address Dependency [ppo:addrpo]

Statement : If b is a store instruction, and there exists some instruction m between a and b in program order such that m has an address dependency on a, then the ordering of a and b must be preserved in the pipeline

Explanation :

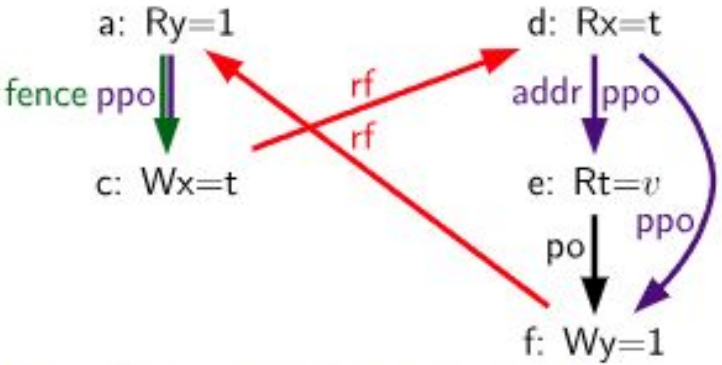
A store cannot be performed at memory until all previous loads that might access the same address have themselves been performed. Such a load must appear to execute before the store, but it cannot do so if the store were to overwrite the value in memory before the load had a chance to read the old value.

Address Dependency: The address used by b or the value it stores depends on the result of a.

Pipeline Ordering: This rule ensures that the store (b) follows the correct order relative to the computation in a, maintaining the proper sequence in the pipeline.

Example of rule 13:

Hart 0		Hart 1	
			li t1, 1
(a)	lw a0,0(s0)	(d)	lw a1, 0(s1)
(b)	fence rw,rw	(e)	lw a2, 0(a1)
(c)	sw s2,0(s1)	(f)	sw t1, 0(s0)
Outcome:		a0=1, a1=t	



Because of the address dependency from (d) to (e), (d) also precedes (f) (outcome forbidden)

Consider Figure 1.16: (f) cannot be executed until the address for (e) is resolved, because it may turn out that the addresses match; i.e., that `a1=s0`. Therefore, (f) cannot be sent to memory before (d) has executed and confirmed whether the addresses do indeed overlap.

THANK YOU

Presented By: S.Lochani vilehya

Emp ID:43317